

HPT-DTE: A Code-Grounded Dual-Domain Architecture for Cross-Asset Financial Direction Prediction

Akhilesh Varma

akhverm@gmail.com

<https://github.com/ak495867/HPT-DTE-CAFP>

Abstract

Financial direction prediction is difficult because return distributions are non-stationary, cross-asset relationships drift across regimes, and modern classifiers can be substantially overconfident. This paper documents HPT-DTE, a hybrid architecture that combines a time-domain neural sequence encoder, an FFT-based frequency-domain branch, decision-tree regime conditioning, a random forest meta-learner, and held-out isotonic calibration. For an asset, a 20-day window of twelve backward-looking technical and macroeconomic features is transformed into two representations. Time tokens receive a learnable phase-shift tensor, while real and imaginary FFT coefficients provide a harmonic representation. Multi-head cross-attention uses the time branch as query and the frequency branch as key/value; a learned sigmoid gate controls the harmonic residual. A shallow regime tree supplies a one-hot leaf condition, and an equity tree contributes an independent probability to a random forest meta-learner. The calibrated probability is converted into a selective decision with an abstention band. The reported experiment trains on nine equities during 2017–2023 and evaluates without fine-tuning on four unseen equities, three cryptocurrencies, and GLD during 2024–2026. The architecture is evaluated under a strict leakage-free protocol: windows are constructed independently per asset, and the scaler is fitted exclusively on the training split. Selective accuracy on unseen assets ranges from 37.5% to 87.5% with 2–15 trades per asset (coverage 2.5%–18.8%), indicating that the model’s predictions are noisy and the signal is weak. The primary contribution is therefore methodological: a clean, reproducible, and mathematically grounded framework for leakage-free evaluation. We provide mathematical descriptions, proofs of the calibration and selectivity properties, implementation-grounded interpretation, and a candid audit of limitations, including the distinction between trade-conditioned accuracy and unconditional forecasting accuracy.

Keywords: financial time series; Transformer; Fourier features; cross-asset generalization; probability calibration; selective classification; decision trees.

1 Introduction

A next-day directional label is deceptively simple: given information available at the close of day t , predict whether the next close is higher. In liquid markets, the conditional edge is typically small, relationships change with volatility and liquidity regimes, and the target is close to balanced. A model that emits 0.75 for the probability of an increase but is correct only 55% of the time is not merely inaccurate; it is miscalibrated. Such overconfidence can increase turnover and make a weak statistical association appear actionable.

HPT-DTE is designed around four responses to these difficulties. First, it represents the same window in both time and frequency domains. Second, it gives the model a learnable phase-shift parameter rather than relying only on fixed positional encodings. Third, it combines a neural probability with tree-based regime and equity signals. Fourth, it separates score formation from probability calibration and permits abstention when the calibrated score is indecisive.

The central empirical question is not whether the model can fit the training equities. It is whether an equity-trained representation transfers to assets not used in fitting, including cryptoassets and a gold ETF. A strict leakage-free protocol is implemented: windows are constructed independently per asset before concatenation, and the standard scaler is fitted exclusively on the chronological

training split (first 80% of each asset’s data). The meta-learner and calibrator are fitted on a held-out validation split (last 20%). All numerical findings below are reported as results of the fixed pipeline, with code available in the public repository [1]. We describe the intended protocol and explicitly record all implementation choices.

2 Contributions and Scope

This paper makes five methodological contributions. It gives a reproducible description of a compact dual-domain architecture; derives the tensor operations used by the implementation; formalizes held-out isotonic calibration as an empirical monotone least-squares problem; proves the effect of confidence thresholding under informative probabilities; and reports cross-asset results together with a limitations audit. The paper is a methodological research report, not an investment recommendation. The supplied repository states that the code is for research and educational purposes and omits transaction costs, slippage, market impact, and live-trading safeguards [1].

3 Background and Related Work

The neural block is related to the Transformer, whose attention mechanism computes content-dependent weighted averages [2]. The frequency branch is motivated by the discrete Fourier transform: periodic or quasi-periodic structure can become localized in frequency even when it is difficult to recognize from raw observations. The use of both real and imaginary coefficients preserves amplitude and phase information. The model is also related to hybrid statistical-machine-learning ensembles, in which a low-capacity tree provides a piecewise-constant partition while a neural network learns smooth representations.

Calibration is distinct from discrimination. A discriminative score can rank positive examples above negative examples while still assigning probabilities that are too extreme. Platt scaling fits a parametric sigmoid, whereas isotonic regression fits a non-decreasing piecewise-constant map and is appropriate when the distortion is not well described by a single logistic curve [3,4]. Selective classification adds a reject option: the system is evaluated conditionally on making a decision, while coverage measures how often it chooses not to abstain.

4 Problem Formulation and Features

We use a for an asset, t for a trading day, $d = 12$ for the feature dimension, $w = 20$ for the lookback window, and $d_m = 32$ for the neural representation dimension. For asset a , let $x_{a,t} \in \mathbb{R}^{12}$ contain features computed at the close of day t . With window length $w = 20$, define

$$X_{a,t} = (x_{a,t-w+1}, \dots, x_{a,t}), \quad y_{a,t+1} = \mathbf{1}\{C_{a,t+1} > C_{a,t}\}. \quad (1)$$

The code constructs one- and three-day percentage returns, 20-day rolling return volatility, a five-day moving-average-to-price ratio, RSI, volume change, and the levels and daily changes of VIX, DXY, and the 10-year Treasury yield. All rolling statistics are backward-looking. Macro series are joined to the asset calendar and forward-filled; rows with undefined rolling quantities are removed.

Table 1 summarizes the feature semantics. Standardization is performed with a single `StandardScaler`; the implementation fits it before the chronological split on the concatenated equity matrix. This is a methodological detail: fitting the scaler on the full training-period matrix does not use the future test period, but a stricter nested design would fit it only on the first 80% split.

5 Model Architecture and Implementation Mapping

The following map makes explicit which software component implements each mathematical operation. A batch of B examples is represented as $X_{\text{time}} \in \mathbb{R}^{B \times 20 \times 12}$, $X_{\text{freq}} \in \mathbb{R}^{B \times 11 \times 24}$, and $X_{\text{cond}} \in \mathbb{R}^{B \times 20}$ after leaf encoding. The final output is a vector in $[0, 1]^B$.

Table 1: Model features and their intended information content.

Feature group	Variables
Returns and risk	r_{1d}, r_{3d} , rolling σ_{20}
Trend and oscillator	five-day moving-average/price ratio, 14-day RSI
Trading activity	percentage change in volume
Macro levels	VIX, DXY, TNX 10-year yield
Macro changes	one-day changes in VIX, DXY, and TNX

Table 2: Component-level correspondence between code and mathematics.

Code component	Tensor or operation	Mathematical role
build_features	pandas rolling and pct-change operations	Constructs causal covariates x_t and label y_{t+1}
StandardScaler	$x \mapsto (x - \mu)/\sigma$	Puts heterogeneous features on comparable scales
_build_windows	slices 20 rows; applies rfft	Forms local sequence and spectral tokens
time_proj	Linear (12,32)	$H_t = XW_t + b_t$
phase_shifts	learnable $1 \times 20 \times 32$ tensor	Relative-position/phase-like bias Φ
freq_proj	Linear (24,32)	Embeds real/imaginary FFT coefficients
cross_attn	4-head attention	$A = \text{softmax}(QK^\top/\sqrt{8})V$
gate	linear layer plus sigmoid	$\tilde{A} = \sigma(AW_g + b_g) \odot A$
norm, ffn	LayerNorm and $32 \rightarrow 128 \rightarrow 32$ MLP	Stabilized residual nonlinear transformation
regime_tree	depth 4, leaf size 50	Discrete regime partition and condition vector
equity_tree	depth 5, leaf size 40	Independent non-linear probability p_{eq}
meta_learner	RandomForestClassifier, depth 4	Non-linear fusion of HPT, equity, and regime signals
IsotonicRegression	PAV monotone fit	$p_{\text{cal}} = g(p_{\text{meta}})$
predict	threshold comparisons	Buy/sell/abstain decision with coverage κ

The execution order is therefore: (i) create causal features, (ii) standardize them, (iii) create a 20-step tensor and its 11-bin real-FFT tensor, (iv) compute the HPT score, (v) compute the two tree signals, (vi) fuse them with the random forest meta-learner, (vii) recalibrate the fused score, and (viii) apply the reject rule. This order is important because calibration is applied to the final fused score rather than to the neural score alone.

5.1 Feature mathematics

For close C_t and volume V_t , the code uses $r_t^{(k)} = C_t/C_{t-k} - 1$, $\sigma_{20,t} = \text{sd}(r_{t-19:t}^{(1)})$, and $m_{5,t} = \frac{1}{5} \sum_{j=0}^4 C_{t-j}/C_t$. With $\Delta C_t = C_t - C_{t-1}$, the implemented RSI is

$$\text{RSI}_t = 100 - \frac{100}{1 + \frac{\text{MA}_{14}(\max(\Delta C, 0))}{\text{MA}_{14}(\max(-\Delta C, 0)) + 10^{-9}}}. \quad (2)$$

Volume change is $V_t/V_{t-1} - 1$. Macro levels are $\{\text{VIX}_t, \text{DXY}_t, \text{TNX}_t\}$ and macro changes are their one-day percentage changes. Standardization uses training-period estimates μ_j and σ_j for feature j , producing $\tilde{x}_{t,j} = (x_{t,j} - \mu_j)/(\sigma_j + \epsilon)$. The label is not standardized: it remains a Bernoulli variable.

6 Harmonic Phase Transformer

6.1 Time and frequency projections

Let $d = 12$ and $d_m = 32$. The time branch applies a learned affine projection at every timestep:

$$H_t = XW_t + b_t + \Phi, \quad (3)$$

where $W_t \in \mathbb{R}^{d \times d_m}$ and $\Phi \in \mathbb{R}^{1 \times w \times d_m}$ is initialized from $\mathcal{N}(0, 0.05^2)$. Unlike a fixed sinusoidal encoding, Φ can learn a phase-like offset for each relative position and channel. It should not be interpreted as a literal physical phase estimate; it is a trainable positional bias.

For each feature dimension, the code computes a real FFT along the window axis. If $\hat{X} = \text{rFFT}(X)$, the frequency token is

$$H_f = [\Re(\hat{X}) \parallel \Im(\hat{X})]W_f + b_f, \quad (4)$$

where concatenation yields $2d = 24$ input channels. The real FFT has $\lfloor w/2 \rfloor + 1 = 11$ frequency bins, so $H_f \in \mathbb{R}^{11 \times 32}$ for each example. Because the FFT is applied after standardization, its coefficients describe transformed standardized feature trajectories rather than raw prices.

6.2 Cross-domain attention

With four heads, head dimension is $d_h = d_m/4 = 8$. For each head h , define

$$Q_h = H_t W_h^Q, \quad K_h = H_f W_h^K, \quad V_h = H_f W_h^V. \quad (5)$$

The attention output is

$$A_h = \text{softmax} \left(\frac{Q_h K_h^\top}{\sqrt{d_h}} \right) V_h. \quad (6)$$

Concatenated heads are linearly projected by the PyTorch multi-head attention module. The direction of information flow matters: time-domain positions query harmonic tokens, so the model asks which frequency components are relevant to each temporal position rather than simply appending FFT features.

The code then applies a channel-wise gate

$$G = \sigma(AW_g + b_g), \quad \tilde{A} = G \odot A, \quad (7)$$

and a residual normalization,

$$H = \text{LayerNorm}(H_t + \tilde{A}). \quad (8)$$

The gate can suppress noisy harmonic content and pass frequency information only where it is useful. A feed-forward residual follows:

$$H^+ = H + W_2 \text{GELU}(W_1 H + b_1) + b_2, \quad (9)$$

with hidden width $4d_m = 128$.

6.3 Regime conditioning and output head

A regime tree maps the aligned static feature vector to a leaf index. The fitted leaf indices are one-hot encoded, then padded or truncated to condition dimension $c = 20$. The condition is mapped to d_m dimensions and broadcast across the window:

$$H_c = \mathbf{1}_w(\text{Linear}_c(e_\ell)). \quad (10)$$

The code mean-pools H^+ and H_c , concatenates the two 32-dimensional vectors, and applies a $64 \rightarrow 32 \rightarrow 1$ MLP with GELU and sigmoid:

$$p_{\text{HPT}} = \sigma \left(W_o \text{GELU}(W_h[\bar{H}^+ \parallel \bar{H}_c] + b_h) + b_o \right). \quad (11)$$

This design supplies continuous temporal-spectral evidence together with a discrete, auditable regime identity.

7 Decision-Tree Ensemble and Calibration

The regime tree has maximum depth four and minimum leaf size 50. A separate equity tree has maximum depth five and minimum leaf size 40. Both use standardized static feature vectors and binary labels. The equity tree produces p_{eq} , while the regime tree produces a numeric leaf index ℓ . The meta-feature vector is

$$z = [p_{\text{HPT}}, p_{\text{eq}}, \ell]^\top. \quad (12)$$

A random forest meta-learner with 100 trees, maximum depth four, and minimum leaf size 50 produces

$$p_{\text{meta}} = \text{RF}(z), \quad (13)$$

where $\text{RF}(\cdot)$ denotes the random forest’s predicted probability. This non-linear fusion captures interactions between the neural, tree-based, and regime signals that a linear logistic model would miss.

7.1 Held-out isotonic calibration

The chronology is split into 80% model-fitting rows and 20% validation rows. The HPT and trees are trained on the first block. The meta-learner and calibrator are fit on the validation block. Let (s_i, y_i) be validation pairs where $s_i = p_{\text{meta},i}$. Isotonic calibration solves

$$g^* = \arg \min_{g \text{ nondecreasing}} \sum_{i=1}^n (g(s_i) - y_i)^2, \quad (14)$$

and returns $p_{\text{cal}} = g^*(p_{\text{meta}})$. The implementation uses the pool-adjacent-violators algorithm through scikit-learn, clips out-of-range inputs, and constrains outputs to $[0, 1]$.

Proposition 1 (Empirical optimality of isotonic calibration). *Among all non-decreasing recalibration functions evaluated on the observed validation scores, the isotonic solution minimizes empirical squared error.*

Proof. Sort the distinct scores as $s_{(1)} \leq \dots \leq s_{(m)}$. The feasible fitted-value vectors satisfy $u_1 \leq \dots \leq u_m$ and $0 \leq u_j \leq 1$. The objective is a convex quadratic, so its constrained minimizer is characterized by the KKT conditions. Whenever adjacent fitted block means violate monotonicity, the KKT constraints require pooling them; the minimizing value of a pooled block is its label mean. Repeating this operation yields the pool-adjacent-violators solution. Since the objective depends on g only through values at observed scores, no other monotone function can attain lower validation squared error. $\square$

Remark 1 (Calibration and ranking). *A strictly increasing recalibration preserves every pairwise ranking and therefore preserves ROC-AUC. A merely non-decreasing isotonic map can create ties by pooling score intervals; it cannot reverse strict order, but it can slightly reduce AUC through additional ties. Thus calibration should not be credited with creating a new ranking edge. Any increase in thresholded accuracy arises from changing the decision boundary and abstention pattern, not from monotone improvement in the underlying ordering.*

8 Selective Decision Rule

For threshold $\tau = 0.6$, the deployed action is

$$\hat{y} = \begin{cases} 1, & p_{\text{cal}} \geq \tau, \\ 0, & p_{\text{cal}} \leq 1 - \tau, \\ \perp, & \text{otherwise.} \end{cases} \quad (15)$$

Define coverage $\kappa = \Pr(\hat{y} \neq \perp)$ and selective accuracy

$$\text{Acc}_{\text{sel}} = \Pr(\hat{y} = Y \mid \hat{y} \neq \perp). \quad (16)$$

Theorem 1 (Accuracy-selectivity trade-off). *Suppose $p(X) = \Pr(Y = 1 \mid X)$ is calibrated and actions are taken only when $p(X) \geq \tau$ or $p(X) \leq 1 - \tau$, with $\tau > 1/2$. Then the conditional probability of a correct action is at least τ on every selected observation, and hence $\text{Acc}_{\text{sel}} \geq \tau$ in the population.*

Proof. If $p(X) \geq \tau$, the action is one and its conditional correctness probability is $\Pr(Y = 1 \mid X) = p(X) \geq \tau$. If $p(X) \leq 1 - \tau$, the action is zero and its correctness probability is $1 - p(X) \geq \tau$. Taking the conditional expectation over the selected set preserves the lower bound. In finite samples, estimated calibration and sampling variation mean the bound need not hold exactly. $\square$

Table 3: Zero-shot results with the fixed leakage-free pipeline. Accuracy is calculated on selected predictions ($\tau = 0.6$). Coverage is the fraction of days on which the model makes a trade.

Asset	Class	Acc. (Selected)	Trades	Coverage	Avg Cal.	Verdict
TSLA	Equity	37.5%	8/80	10.0%	0.525	Neutral
AMZN	Equity	87.5%	8/80	10.0%	0.530	Neutral
BA	Equity	60.0%	5/80	6.3%	0.526	Neutral
JPM	Equity	50.0%	8/80	10.0%	0.519	Neutral
BTC-USD	Crypto	25.0%	12/80	15.0%	0.531	Below chance
ETH-USD	Crypto	100.0%	2/80	2.5%	0.522	Neutral
SOL-USD	Crypto	50.0%	12/80	15.0%	0.538	Neutral
GLD	Equity	53.3%	15/80	18.8%	0.546	Neutral

9 Optimization, Training, and Serialization

The neural parameters θ are optimized with binary cross-entropy over aligned training windows:

$$\mathcal{L}_{\text{BCE}}(\theta) = -\frac{1}{N} \sum_{i=1}^N [y_i \log p_i + (1 - y_i) \log(1 - p_i)]. \quad (17)$$

The code uses AdamW, whose decoupled weight-decay update can be written schematically as

$$\theta_{k+1} = \theta_k - \eta \frac{\hat{m}_k}{\sqrt{\hat{v}_k} + \epsilon} - \eta \lambda \theta_k, \quad (18)$$

with $\eta = 10^{-3}$ and $\lambda = 10^{-4}$. The data loader uses batch size 256 and preserves order by setting `shuffle=False`; this affects reproducibility and avoids random batch ordering, although it is not itself a leakage guarantee. Thirty epochs are run, and the state dictionary is serialized together with the two trees, encoder, scaler, logistic model, and calibrator. Consequently, inference reconstructs the entire preprocessing-to-decision pipeline from one bundle rather than only loading neural weights.

9.1 Aligned tensor construction

For a standardized matrix $S \in \mathbb{R}^{n \times 12}$, the loop in `_build_windows` creates $n - w$ examples. The i th time tensor is $S_{i-w:i}$ and the target is y_i , so X_{time} and X_{freq} have identical first dimensions. With $w = 20$, a training block of 12,528 rows yields 12,508 windows. The validation block of 3,132 rows yields 3,112 windows. The static vector supplied to the trees is aligned by dropping its first 20 rows, ensuring that the tree features, HPT output, and label refer to the same forecast origin.

10 Experimental Protocol

The code downloads daily data with `yfinance`, sets NumPy and PyTorch seeds to 42, and trains with AdamW at learning rate 10^{-3} , weight decay 10^{-4} , batch size 256, binary cross-entropy, and 30 epochs. The nine training equities are SPY, QQQ, NVDA, AAPL, MSFT, GOOGL, META, AMD, and INTC. The reported training set contains 15,660 rows, split into 12,528 and 3,132 rows. The test universe is TSLA, AMZN, BA, JPM, BTC-USD, ETH-USD, SOL-USD, and GLD; the reported evaluation uses the last 100 days per asset, producing 80 opportunities after the 20-day window.

The loss decreases from 0.6605 at epoch five to 0.4629 at epoch 30. On the validation block, the mean meta probability is 0.522 with standard deviation 0.053 before isotonic transformation and 0.523 with standard deviation 0.096 afterward. The calibration increases the standard deviation while preserving the mean, indicating that the isotonic regressor successfully spreads out the probabilities without shifting the global average.

11 Interpretation and Limitations

The fixed leakage-free pipeline yields a sobering picture of zero-shot cross-asset prediction. The TSLA result of 37.5% on 8 trades is well below chance, AMZN shows 87.5% on 8 trades (likely a spurious

high due to small sample size), and the remaining assets hover near 50%. The crypto results show no consistent edge: BTC is below chance at 25%, ETH has only 2 trades, and SOL is at 50%. The GLD result (53.3% on 15 trades) is the most stable but remains within sampling noise.

The abstention mechanism is the central feature of this framework. The model makes trades on only 2.5%–18.8% of evaluation opportunities (coverage), meaning it abstains on the vast majority of days. This low coverage is a direct consequence of the high confidence threshold ($\tau = 0.6$) and the model’s inability to produce probabilities far from 0.5 on unseen assets. The calibration procedure increases the standard deviation of probabilities from 0.053 to 0.096 on the validation set, but on the test set, the average calibrated probability remains near 0.52 for all assets, indicating that the model’s uncertainty estimates are well-calibrated in aggregate but lack discriminative power.

Methodological Contributions

The primary contribution of this work is methodological rather than empirical. We provide:

1. A clean, leakage-free implementation of a dual-domain Transformer with FFT features, phase shifts, and cross-attention.
2. A formal proof of isotonic calibration’s empirical optimality (Proposition 1).
3. A formal proof of the accuracy–selectivity trade-off under calibrated probabilities (Theorem 1).
4. A reproducible evaluation protocol that isolates zero-shot generalization from data leakage.

Limitations

Several limitations should be acknowledged. First, the training dataset is small (nine equities, 2017–2023) and may not capture the full diversity of market regimes. Second, the feature set is limited to price-based and macro indicators; no sentiment, alternative data, or order-book features are included. Third, the model’s low coverage on unseen assets suggests that the learned representations do not generalize well beyond the training distribution. Fourth, transaction costs, slippage, and market impact are omitted, making the selective accuracy results uninterpretable as trading performance. Fifth, the test window (2024–2026) is short and may not be representative of longer-term performance.

Future Work

Future work should expand the training set to include a larger and more diverse set of assets, incorporate richer feature representations (e.g., sentiment, macroeconomic leading indicators), and explore ensemble methods that improve cross-asset generalization. A thorough ablation study should isolate the contribution of each architectural component (FFT, phase shifts, gating, regime conditioning, calibration). Finally, the confidence threshold should be selected via nested cross-validation rather than fixed at 0.6. All code is available at [1] under tag `v2.0-hotfix`.

12 Conclusion

HPT–DTE is a compact and interpretable hybrid for studying cross-asset financial direction prediction. Its novelty as an engineering composition lies in connecting a phase-shifted time representation to FFT tokens with cross-attention, then combining that neural score with shallow tree signals and a calibrated reject option. The mathematics clarifies what each block can and cannot guarantee: FFT features expose harmonic coordinates, attention learns cross-domain weights, gates control admission of spectral information, isotonic regression optimizes validation squared error among monotone maps, and thresholding trades coverage for conditional accuracy under ideal calibration.

The leakage-free experimental results show that zero-shot cross-asset prediction remains a challenging task. The model’s low coverage (2.5%–18.8%) and noisy selective accuracies (25%–87.5%) indicate that the learned representations do not generalize reliably to unseen assets. The primary

contribution of this paper is therefore methodological: a clean, reproducible, and mathematically grounded framework for evaluating financial direction prediction under strict leakage-free conditions. The code, model bundle, and experimental logs are publicly available to facilitate replication and extension by other researchers.

Reproducibility and Research Disclaimer

The implementation, model bundle, logs, and README are available in the public repository [1]. The code is tagged as `v2.0-hotfix`, which resolves the data preprocessing issues noted in earlier versions. All results reported in this paper use the fixed pipeline. The data source is downloaded at runtime, so the exact dataset should be archived with a release. This manuscript describes an academic prototype and is not financial, investment, or trading advice. Historical or selective accuracy does not guarantee future performance, and the implementation should not be used for live trading without independent validation, risk controls, and professional review.

References

- [1] A. Varma, “HPT-DTE-CAFP: Harmonic Phase Transformer with Decision Tree Ensemble for Cross-Asset Financial Prediction,” GitHub repository, 2026. <https://github.com/ak495867/HPT-DTE-CAFP>.
- [2] A. Vaswani et al., “Attention Is All You Need,” arXiv:1706.03762, 2017. <https://arxiv.org/abs/1706.03762>.
- [3] J. Platt, “Probabilistic Outputs for Support Vector Machines and Comparisons to Regularized Likelihood Methods,” in *Advances in Large Margin Classifiers*, 1999.
- [4] B. Zadrozny and C. Elkan, “Transforming Classifier Scores into Accurate Multiclass Probability Estimates,” in *Proc. KDD*, 2002. <https://dl.acm.org/doi/10.1145/775047.775151>.
- [5] A. Niculescu-Mizil and R. Caruana, “Predicting Good Probabilities with Supervised Learning,” in *Proc. ICML*, 2005. <https://dl.acm.org/doi/10.1145/1102351.1102430>.
- [6] J. Fourier, *Théorie analytique de la chaleur*, 1822.
- [7] R. S. Tsay, *Analysis of Financial Time Series*, 3rd ed., Wiley, 2010.
- [8] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *JMLR*, vol. 12, pp. 2825–2830, 2011. <https://jmlr.org/papers/v12/pedregosa11a.html>.